

tensormodels: Statistical Models for Tensor-Valued Data

by Anupama Kannan, David Kahle, and Michael Gallagher

Abstract *tensormodels* is an R package for simulation, maximum likelihood estimation, and model diagnostics for tensor-variate distributions arising in multiway array data. The package implements the tensor-variate normal distribution and several normal variance–mean mixture families, including the skew-t, generalized hyperbolic, variance gamma, and normal inverse Gaussian distributions. Estimation is performed via likelihood-based and expectation–maximization algorithms under a Kronecker-separable covariance model, yielding mode-specific covariance estimates through iterative updates. Model assessment tools include distance-based diagnostics, such as Mahalanobis distance plots constructed from tensor-whitened residuals. In addition, *tensormodels* provides implementations of widely used tensor decompositions for approximation and dimension reduction, including Tensor-Train decomposition via TT-SVD, Higher-Order Singular Value Decomposition (HOSVD), and Parallel Factor Analysis (CP/PARAFAC) computed using alternating least squares. It includes common linear algebra and tensor operations, such as matricization, the n-mode products, and the Khatri-Rao product.

1 Introduction

Lately, with the rise of data measurement tools and big data, the creation of higher dimensional data has become more and more common. These unique types of data require corresponding unique analytical tools to properly fit models with interpretation. Many researchers default to vectorizing or matricizing this higher order data to fit models that they are comfortable with and have pre-established tools. Additionally, they will commonly assume that data satisfies the assumptions of normality, which can sometimes be a very extreme choice that is only made to make computation and analysis easier.

The package **tensormodels** aims to assist with both of these issues. Firstly, it contains the tensor variate normal distribution and several skewed distributions under the normal mean-variance family model that can model a multitude of different shapes of data. Secondly, it contains diagnostic test and model heuristics to pick the best distribution for the user’s data set. It also contains random number generation and density functions for researchers who merely want to better understand tensor variate distributions.

In the following sections, we will explain the theory behind tensor variate distributions, compare related tensor software across several programming languages, explain some unique choices in the package design, show a simulation set up, and apply the models to a real-world problem.

2 Background and related work

The tensor space is still being established, with works using conflicting notation. In this section, we will set a standard for certain operations, and explain the basic underlying theory for several of the functions. Tensors are a generalization of vectors and matrices, allowing us to represent higher-order data into a structure. We need new terminology to describe these objects.

2.1 Order

The order of a tensor is the number of indices needed to identify an entry. This is the same as the number of values needed inside R’s subsetting brackets to pull one element out. A

vector is a first-order tensor, a matrix is a second-order tensor, and any object beyond that is a higher-order tensor.

For example, take the vector $\mathbf{a} = (1, 2, 3)$. To extract a specific element, you only require one value, so it has an order 1.

```
a <- c(1, 2, 3)
```

```
a[2]
#> [1] 2
```

The matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

has order two, since its entries are indexed by row and column.

```
A <- matrix(data = 1:6, nrow = 2, ncol = 3)
```

```
A[2, 1]
#> [1] 2
```

A third-order tensor requires three indices. The following tensor has dimensions $4 \times 3 \times 2$, so we need to specify a position along each mode to pull an element.

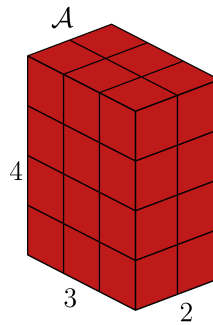


Figure 1: A $4 \times 3 \times 2$ tensor $\mathcal{A}$

```
A <- array(data = 1:24, dim = c(4, 3, 2))
```

```
A[3, 1, 1]
#> [1] 3
```

2.2 Mode

The mode is a specific axis of a tensor. For a matrix, the two modes are usually called rows and columns. However, since a tensor can have three or more dimensions, we now need a term for the specific dimension being referred to.

Consider a tensor $\mathcal{A} \in 4 \times 2 \times 3$. We would say the first mode has dimension 4, the second mode has dimension 2, and the third mode has dimension 3. Note that the modes of a tensor are dependent on the structure of the object. For example, we could keep the same data in $\mathcal{A}$ but permute the modes, leading to a new tensor $\mathcal{B} \in 4 \times 3 \times 2$. When communicating a tensor, it is important to keep the modes consistent and keep track of what each mode represents. For example, the modes of a tensor could represent a specific sensor ID, time of measurement, and spatial location.

2.3 n -mode product

In regular matrix multiplication, we can multiply two matrices as long as the number of columns in the first matrix matches the rows of the second matrix. Now, suppose we want to multiply a tensor and a matrix. Now, we need a new operator to specify which mode from the tensor should line up with the matrix. The n -mode product specifies that the n -th mode of a tensor should have the same size as the columns of a matrix.

Let tensor $\mathcal{A} \in I_1 \times I_2 \times \dots \times I_n$ and let a matrix $\mathbf{U} \in J \times I_{n+1} \times \dots \times I_N$. Then the n -mode product of $\mathcal{A}$ and $\mathbf{U}$ can be defined as

$$(\mathcal{A} \times_n \mathbf{U}) \in I_1 \times \dots \times I_{n-1} \times J \times I_{n+1} \times \dots \times I_N.$$

Note that the modes of the result stay the same as $\mathcal{A}$ other than the n -th mode being replaced with J .

Its entries are

$$(\mathcal{A} \times_n \mathbf{U})_{i_1, \dots, i_{n-1}, j, i_{n+1}, \dots, i_N} = \sum_{i_n=1}^{I_n} a_{i_1, i_2, \dots, i_N} u_{j, i_n}.$$

The tensor and matrix share one mode in common, denoted here as I_n . This operation is also called the tensor times matrix product.

This function is called `n_prod()` in the package **tenormodels**. It accepts a tensor, a matrix, and a value n specifying which mode to multiply across. For example, let us take $\mathbf{a} \in \mathbb{R}^3$ and $\mathbf{U} \in \mathbb{R}^{2 \times 3}$. Then if we compute $\mathbf{a} \times_1 \mathbf{U}$, we expect the final result to be $\in \mathbb{R}^{2 \times 1}$.

```
a_vec <- c(1, 2, 3)
u_mat <- matrix(4:9, ncol = 3)

n_prod(tensor = a_vec, mat = u_mat, n = 1)
#> [1] 40 46
```

We can also do a more complex example with a tensor and a matrix. Here, $\mathcal{A} \in \mathbb{R}^{2 \times 3 \times 4}$. Then if we compute $\mathcal{A} \times_2 \mathbf{U}$, we expect the final result to be $\in \mathbb{R}^{2 \times 2 \times 4}$.

```
A_tensor <- array(1:24, dim = c(2, 3, 4))

dim(n_prod(A_tensor, u_mat, n = 2))
#> [1] 2 2 4
```

In **tenormodels**, the n -mode product is used internally to apply covariance transformations one mode at a time. For random generation, independent standard normal tensors are multiplied by Cholesky factors along each mode to induce the desired covariance structure. For density evaluation and model diagnostics, centered tensors are multiplied by inverse covariance factors along each mode, producing standardized tensor residuals and Mahalanobis distances.

2.4 Matricization

Matricization, also called unfolding, collapses a tensor into a matrix. It accepts an argument k that determines which mode to place in the rows, and then combines every other mode into the columns. This allows us to focus on variation along a specific mode.

Let $\mathcal{A} \in I_1 \times \dots \times I_N$ be an N -th order tensor. The k -th matricization of $\mathcal{A}$ is a matrix $\mathbf{A}_k \in I_1 I_2 \dots I_k \times I_{k+1} \times I_{k+2} \times \dots \times I_N$, whose elements are taken column-wise from $\mathcal{A}$. The elements are $\mathbf{A}_{(k)}(i_1, \dots, i_k, i_{k+1}, \dots, i_N) = \mathcal{A}(i_1, \dots, i_k, i_{k+1}, \dots, i_N)$.

In **tenormodels**, this function is called `matricization()`. It accepts a tensor and k .

Example

The slices of A along the 3rd mode are two 4×3 matrices.

$$A_1 = \begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 13 & 17 & 21 \\ 14 & 18 & 22 \\ 15 & 19 & 23 \\ 16 & 20 & 24 \end{bmatrix}.$$

Let us say we want to matricize along the first mode. This compresses the second and third mode into the columns of a matrix, so we would expect to obtain a matrix $\in \mathbb{R}^{4 \times 6}$. We will do this by moving the values in the third mode into the columns.

The notation for matricization along the first mode is $A_{(1)}$.

$$A_{(1)} = \begin{bmatrix} 1 & 5 & 9 & 13 & 17 & 21 \\ 2 & 6 & 10 & 14 & 18 & 22 \\ 3 & 7 & 11 & 15 & 19 & 23 \\ 4 & 8 & 12 & 16 & 20 & 24 \end{bmatrix}$$

```
A <- array(1:24, dim = c(4, 3, 2))
```

```
matricization(A, k = 1)
#>      [,1] [,2] [,3] [,4] [,5] [,6]
#> [1,]    1    5    9   13   17   21
#> [2,]    2    6   10   14   18   22
#> [3,]    3    7   11   15   19   23
#> [4,]    4    8   12   16   20   24
```

In **tenso**models, matricization is used to initialize and update mode-specific covariance matrices through ordinary matrix crossproducts.

2.5 Tensor variate distributions

There are several tensor variate distributions that are generalizations of matrix variate distributions. This section introduces the distributions that are used by **tenso**models.

The tensor variate normal distribution can be considered a generalization of the multivariate and matrix variate normal distributions.

Univariate Normal

First, consider a standard univariate normal draw $Z \sim N(0, 1)$. If we shift and scale it to generate $X = \mu + \sigma Z$, then $X \sim N(\mu, \sigma^2)$. That is, an affine transformation of a standard univariate normal draw describes all possible univariate normal distributions.

Multivariate Normal

That generated one element. Now let $\mathbf{Z}$ be a $p \times 1$ vector containing IID standard normal draws. Then $\mathbf{Z} \sim N_p(\mathbf{0}, \mathbf{I}_p)$. If we let $\mathbf{A}\mathbf{A}^T = \Sigma$, then define $\mathbf{X} = \bar{\mathbf{x}} + \mathbf{A}\mathbf{Z}$. Now $\mathbf{X} \sim N_p(\bar{\mathbf{x}}, \Sigma)$. That is, $\mathbf{X}$ defines all the possible multivariate normal distributions.

Matrix Variate Normal

Let $\mathbf{Z}$ be a $p \times q$ matrix with IID standard normal entries. Choose matrices $\mathbf{A}$ and $\mathbf{B}$ such that $\mathbf{U} = \mathbf{A}\mathbf{A}^T$, $\mathbf{V} = \mathbf{B}\mathbf{B}^T$. Now define $\mathbf{X} = \mathbf{M} + \mathbf{A}\mathbf{Z}\mathbf{B}^T$. We say that $\mathbf{X} \sim N_{pq}(\mathbf{M}, \mathbf{U}, \mathbf{V})$. That is, $\mathbf{X}$ defines all the possible matrix variate normal distributions.

Table 1: Mixing distributions used to construct the skewed tensor variate distributions.

Distribution	Distribution of W	Parameters	Parameter support
Tensor variate skewed-t	Inv-Gamma($\nu/2, \nu/2$)	ν	$\nu > 0$
Tensor variate generalized hyperbolic	Gen Inv Gaussian($\omega, 1, \lambda$)	λ, ω	$\omega > 0, \lambda \in \mathbb{R}$
Tensor variate variance gamma	Gamma(γ, γ)	γ	$\gamma > 0$
Tensor variate normal inverse Gaussian	Inv Gaussian($1, \kappa$)	κ	$\kappa > 0$

Tensor Variate Normal

Let $\mathcal{Z}$ be an order- D tensor with IID standard normal entries and $\Sigma_1, \dots, \Sigma_D$ be positive definite matrices describing the variance-covariance structure along each mode. For each mode, choose $\mathbf{A}_j$ such that $\Sigma_j = \mathbf{A}_j \mathbf{A}_j^T$, $j = 1, \dots, D$.

Then we can define $\mathcal{X}$ as $\mathcal{X} = \mathcal{M} + \mathcal{Z} \times_1 \mathbf{A}_1 \times_2 \mathbf{A}_2 \cdots \times_D \mathbf{A}_D$. That is, $\mathcal{X}$ defines all the possible tensor variate normal distributions.

Skewed distributions as transformations of the tensor variate normal

To generate tensor variate skewed distributions, we use the normal variance mean mixture model (Barndorff-Nielsen et al., 1982). An r -dimensional random vector x is a normal variance-mean mixture with mixing distribution F if, for a given $u \geq 0$ that follows a probability distribution F on $[0, \infty)$, $x|u \sim N_r(\mu + u\beta, u\Sigma)$. $\Sigma \in \mathbb{R}^{r \times r}$ is a constant, positive-definite matrix and $\mu \in \mathbb{R}^r$ and $\beta \in \mathbb{R}^r$ are constant vectors.

We state that x is a normal variance-mean mixture with position μ , drift β , structure matrix Σ , and mixing distribution F . Different mixing distributions lead to different distributions. For example, if F , the mixing distribution, is the generalized inverse Gaussian distribution, then the resulting distribution of x is a generalized hyperbolic distribution.

In the tensor variate case, the normal variance mean mixture model is an efficient way to introduce skewness. A random tensor $\mathcal{X}$ can be written as

$$\mathcal{X} = \mathcal{M} + W\mathcal{A} + \sqrt{W}\mathcal{V}, \quad (1)$$

where $\mathcal{M}$ is a location tensor, $\mathcal{A}$ is a skewness tensor, $\mathcal{V} \sim \mathcal{N}_n^*(\mathbf{O}, \bigotimes_{d=1}^D \Sigma_d)$ and $W > 0$ is a positive random variable. The distribution of $\mathcal{X}$ will depend on the distribution of W .

The mixing distributions and their parameters for each of the skewed tensor variate distributions are summarized below. In each case, the support of W is $(0, \infty)$.

The tensor variate skewed-t distribution can be generated by setting $W \sim \text{Inv-Gamma}(\nu/2, \nu/2)$ in Equation 1. We then say that $\mathcal{X} \sim \text{TVST}_n(\mathcal{M}, \mathcal{A}, \bigotimes_{d=1}^D \Sigma_d, \nu)$.

The tensor variate generalized hyperbolic distribution can be generated by setting $W \sim I(\omega, 1, \lambda)$ in Equation 1. We then say that $\mathcal{X} \sim \text{TVGH}_n(\mathcal{M}, \mathcal{A}, \bigotimes_{d=1}^D \Sigma_d, \lambda, \omega)$.

The tensor variate variance gamma distribution can be generated by setting $W \sim \text{Gamma}(\gamma, \gamma)$ in Equation 1. We then say that $\mathcal{X} \sim \text{TVVG}_n(\mathcal{M}, \mathcal{A}, \bigotimes_{d=1}^D \Sigma_d, \gamma)$.

The tensor variate normal inverse Gaussian distribution can be generated by setting $W \sim \text{IG}(1, \kappa)$ in Equation 1. We then say that $\mathcal{X} \sim \text{TVNIG}_n(\mathcal{M}, \mathcal{A}, \bigotimes_{d=1}^D \Sigma_d, \kappa)$.

These choices of W give the four skewed families implemented in **tensoRmodels**. Their common representation also makes it possible to use a shared estimation framework while changing the distribution of the latent mixing variable.

3 Tensor software across programming languages

There are several existing software packages for tensors in several programming languages, but most emphasize tensor algebra or decompositions over probability models. The unique aspect of **tenso**models is the support for tensor-variate normal and skewed tensor-variate distributions. These include random number generation, density functions, likelihood estimation, and model diagnostics.

One of the most popular softwares is the Tensor Toolbox for MATLAB (Bader et al., 2025; Bader and Kolda, 2006). This toolbox introduces a tensor class to generate dense tensors from a multidimensional MATLAB array. This class object has unique attributes so that the user can use methods such as `size`, `ndims`, `norm`, `unfold`, and `ttm`. It also supports decompositions on the tensor object with methods like `mttkrp`, `cp_als`, `tucker_als`, and `hosvd`. Tensor Toolbox also defines separate intentional classes for other tensor formats: `sptensor` for sparse tensors, `ktensor` for Kruskal/CP factored tensors, and `ttensor` for Tucker factored tensors (Bader and Kolda, 2007).

TensorLy in Python is similar to the Tensor Toolbox for MATLAB. It contains basic tensor operations and common decompositions including CP, Tucker, and Tensor-Train. A major proof of TensorLy is that it has backend support across other common frameworks in Python, including NumPy, PyTorch, and TensorFlow. This makes it easier for the user to run functions on their GPU or along multiple machines (Kossaifi et al., 2019). TensorFlow is another well-known package in Python, but it primarily uses the tensor object as inputs for machine learning methodologies rather than trying to expand on operations and decompositions.

The Python Tensor Toolbox, **pyttb** is a refactoring of the Tensor Toolbox for MATLAB. Thus, it ports the common functions from Tensor Toolbox into Python, including the various unique tensor classes and decompositions (pyttb developers, 2026).

In Julia, **TensorOperations.jl** have support for tensor contractions and **TensorToolbox.jl** is Julia's port of the MATLAB Tensor Toolbox (Periša, 2023; TensorOperations.jl developers, 2026).

In R, **rTensor** has an S4 Tensor class and includes common operations and decompositions. It contains the CP, Tucker, and HOSVD (Li et al., 2024a). **ttTensor** adds the Tensor-Train decomposition to this framework (Li et al., 2024b). **tenso**rr is now archived on CRAN, but it originally provided support for S4 sparse tensor and dense tensor classes.

The comparison also explains the class design adopted in **tenso**models. Packages such as MATLAB Tensor Toolbox and **rTensor** make the tensor class central: users create a tensor object first, and then methods dispatch on that object. **tenso**models uses the base R array so that users do not have to learn the S4 class. This keeps **tenso**models close to ordinary R array workflows.

Table 2: Representative tensor software and tensor object designs across MATLAB, Python, Julia, and R.

Language	Software	Tensor representation	Closest connection to tensormodels
MATLAB	Tensor Toolbox	Dedicated classes: ‘tensor’, ‘sptensor’, ‘ktensor’, and ‘ttensor’.	‘unfold’ and ‘ttm’ correspond closely to ‘matricization()’ and ‘n_prod()’; CP, Tucker, and HOSVD functions match decomposition goals.
MATLAB	Tensorlab	Mostly MATLAB arrays, cells, and structured model descriptions for optimization.	‘cpd’ and ‘lmlra’ overlap with decomposition workflows, but not with tensor-variate density or MLE routines.
Python	TensorLy	Backend tensor arrays with a functional API; the tensor object often comes from NumPy, PyTorch, TensorFlow, JAX, or CuPy.	‘unfold’, ‘mode_dot’, ‘parafac’, ‘tucker’, and ‘tensor_train’ match common tensor operations and decompositions.
Python	pyttb	Python classes modeled after MATLAB Tensor Toolbox: dense, sparse, Kruskal/CP, and Tucker objects.	Class names and decomposition functions are similar to MATLAB Tensor Toolbox and useful for translating algorithms to Python.
Julia	TensorOperations.jl	Julia arrays combined with index-notation macros for tensor contractions.	Contractions and index-aware products are related to tensor algebra, but the focus is not tensor-variate statistical modeling.
Julia	ITensors.jl	Index-aware ‘ITensor’ objects for tensor-network computations.	Useful comparison for indexed tensor classes and tensor networks rather than statistical tensor distributions.
Julia	TensorToolbox.jl	Julia implementation inspired by MATLAB Tensor Toolbox object and function conventions.	Useful point of comparison for Julia users who know the MATLAB Tensor Toolbox style.
R	rTensor	S4 ‘Tensor’ object wrapping an R array.	Closest R neighbor for tensor algebra and decompositions; it uses a heavier S4 class design.
R	ttTensor	Objects and functions specialized to Tensor-Train representations.	Closest R neighbor for the Tensor-Train portion of tensormodels.

Purpose	tensormodels	Similar functions elsewhere
Create or store a dense tensor	Uses base R array object	MATLAB Tensor Toolbox ‘tensor’; TensorLy ‘tensor’; pyttb ‘tensor’; rTensor ‘Tensor’
Represent sparse or factored tensors	No separate sparse/factored tensor class; decomposition results are ordinary lists	MATLAB Tensor Toolbox ‘sptensor’, ‘ktensor’, ‘ttensor’; pyttb equivalents; ITensors.jl ‘ITensor’
Matricize / unfold	‘matricization()’	MATLAB Tensor Toolbox ‘unfold’ / ‘tenmat’; TensorLy ‘unfold’; rTensor unfolding methods
n-mode product	‘n_prod()’, ‘nm_prod()’	MATLAB Tensor Toolbox ‘ttm’; TensorLy ‘mode_dot’; rTensor ‘ttm’; TensorOperations.jl contractions
Khatri-Rao and MTTKRP-style computations	‘khatri_rao()’; used internally by decomposition code	MATLAB Tensor Toolbox ‘khatrirao’ / ‘mttkrp’; TensorLy Khatri-Rao and tenalg routines
CP/PARAFAC	‘cp()’, ‘cp_reconstruct()’	MATLAB Tensor Toolbox ‘cp_als’; Tensorlab ‘cpd’; TensorLy ‘parafac’; rTensor ‘cp’
Tucker / HOSVD	‘tucker()’, ‘hosvd()’, ‘tucker_reconstruct()’, ‘hosvd_reconstruct()’	MATLAB Tensor Toolbox ‘tucker_als’ and ‘hosvd’; Tensorlab ‘lmlra’; TensorLy ‘tucker’; rTensor ‘hosvd’
Tensor-Train	‘tt()’, ‘tt_reconstruct()’	TensorLy ‘tensor_train’; ttTensor ‘TTSVD’, ‘TTCross’, ‘TTWOPT’
Tensor-variate distributions	‘rtnorm()’, ‘dtnorm()’, ‘rtskewt()’, ‘dtskewt()’, ‘rtvargamma()’, ‘dtvargamma()’, ‘rtgenhyper()’, ‘dtgenhyper()’, ‘rtinvgauss()’, ‘dtinvgauss()’, and ‘tensor_mle()’	Limited support in the general tensor software above

4 Package design

The main goal of **tenormodels** is to provide functions for random number generation, density computation, and maximum likelihood estimation for common tensor variate distributions. Additionally, the package provides diagnostic tests for the structure of input data.

Distribution functions follow familiar R naming conventions: names beginning with `r` generate random draws, names beginning with `d` evaluate densities, and `tensor_mle()` fits a selected tensor-variate model. Tensor algebra helpers such as `matricization()`, `n_prod()`, `nm_prod()`, `tensor_prod()`, and `khatrao_rao()` are available to users and are also used internally by the likelihood and decomposition routines. Decomposition functions are grouped with explicit reconstruction helpers: `cp()` and `cp_reconstruct()`, `tucker()` and `tucker_reconstruct()`, `hosvd()` and `hosvd_reconstruct()`, and `tt()` and `tt_reconstruct()`.

One intentional decision was to store independent, identically distributed draws from a tensor variate distribution as a list. For example, if we generate $n = 100$ draws from a tensor variate normal distribution of dimension $2 \times 3 \times 4$, the resulting R object will be a list of length 100. Each element in the list is a base R array object of dimension $2 \times 3 \times 4$. Other packages in this space created unique S3 and S4 class objects to store these types of draws. However, these choices are difficult for the average R user, as they have unique interactions and functions.

Another option would be to output a tensor of size $100 \times 2 \times 3 \times 4$; that is, create a tensor of one higher dimension where the first mode represents the index of the draw. The choice to have the first mode represent the draw is arbitrary and could potentially confuse users, especially if they permute the dimensions.

```
three_dim_iid <- rtnorm(n = 100, mu = array(1:12, dim = c(2, 3, 4)))

length(three_dim_iid)
#> [1] 100
str(three_dim_iid[1:2], max.level = 1)
#> List of 2
#> $ : num [1:2, 1:3, 1:4] 1.52 3.22 2.57 3.85 6.34 ...
#> $ : num [1:2, 1:3, 1:4] -0.0797 1.5062 3.4586 3.817 5.5527 ...
#> - attr(*, "dim")= int 2
```

Model fits are returned as ordinary R lists. For example, a tensor-variate normal fit contains the estimated mean array `mu`, the mode-specific covariance matrices `sigmas`, likelihood-based summaries, and convergence information when relevant. Skewed families add distribution-specific parameters and skewness arrays. This return design keeps the model output easy to inspect with base R tools while avoiding a separate hierarchy of model classes.

4.1 Core workflow

This package contains random number generations for several tensor variate distributions. Below is a simulation from a tensor variate normal of size $2 \times 3 \times 4$. The dimension of the draws is the same as the `mu` object provided. The covariance structure is specified with a list of matrices, where each matrix must be a square of the corresponding mode.

```
mu_true <- array(1:24, dim = c(2, 3, 4))

S1 <- crossprod(matrix(rnorm(4), nrow = 2))
S2 <- crossprod(matrix(rnorm(9), nrow = 3))
S3 <- crossprod(matrix(rnorm(16), nrow = 4))
```



```
sigmas_true <- list(S1, S2, S3)

tvn_draws <- rtnorm(n = 1e3, mu = mu_true,
                   sigmas = sigmas_true)
```

Given a list of draws, the package can estimate the MLEs using the function `tensor_mle()`. The model argument is required to know which distribution to fit to the data.

```
tvn_mles <- tensor_mle(tvn_draws, model = "normal")

str(tvn_mles)
#> List of 6
#> $ mu      : num [1:2, 1:3, 1:4] 0.838 2.514 2.803 4.669 4.983 ...
#> $ sigmas:List of 3
#> ..$ : num [1:2, 1:2] 0.277 -0.383 -0.383 1.723
#> ..$ : num [1:3, 1:3] 1.389 1.416 0.119 1.416 1.597 ...
#> ..$ : num [1:4, 1:4] 17.53 3.23 3.36 4.64 3.23 ...
#> $ loglik: num -8984
#> $ k      : num 41
#> $ AIC    : num 18050
#> $ BIC    : num 18252
```

There are also density functions for each distribution. For the tensor variate normal, the required arguments are `mu` and `sigmas`. There is an optional `log` argument to compute the log density rather than the density itself.

```
dtnorm(tvn_draws[[1]], mu = mu_true, sigmas = sigmas_true)
#> [1] 0.001097981
```

4.2 Tensor decompositions

The idea of decompositions can also apply to tensors. The package provides the Tucker, PARAFAC, higher-order singular value, and Tensor-Train decompositions.

Tucker decomposition

The idea behind the Tucker decomposition is similar to rank factorization for matrices. The Tucker decomposition factorizes an N -th order tensor into a core tensor and a list of matrices ([Hitchcock, 1927](#)):

$$\mathcal{A} = \mathcal{B} \times_1 \mathbf{U}_1 \times_2 \mathbf{U}_2 \cdots \times_N \mathbf{U}_N.$$

The core tensor $\mathcal{B}$ has the same order as the original tensor $\mathcal{A}$, and the number of $\mathbf{U}$ matrices is also the number of modes. The modes of $\mathcal{B}$ are called the Tucker ranks, which determine how reduced the form is.

Each matrix $\mathbf{U}_n$ contains principal factors for mode n . The $\mathbf{U}_n$ is a basis for the n th mode, and thus spans a subspace of $\mathbb{R}^{I_n}$. The core tensor $\mathcal{B}$ represents the strength of interactions between factors. The idea behind this decomposition is to reduce the dimensionality of the input. We are able to summarize the higher-order object $\mathcal{A}$ with a smaller core tensor $\mathcal{B}$ and several factor matrices $\mathbf{U}_1, \mathbf{U}_2, \dots, \mathbf{U}_N$. However, one key issue with the Tucker decomposition is that it is not unique.

PARAFAC decomposition

The PARAFAC decomposition has several other names—CANDECOMP, CP, and Tensor Rank decomposition. PARAFAC stands for parallel factor analysis while CANDECOMP

stands for canonical polyadic decomposition. This decomposition is a special case of the Tucker decomposition, where the core tensor is superdiagonal.

Let us say we want to approximate the tensor $\mathcal{A} \in I_1 \times I_2 \times \dots \times I_N$ with a linear combination of R rank 1 tensors. The approximation can be written as

$$\mathcal{A} \approx \sum_{r=1}^R \lambda(r) \mathbf{u}_1(:, r) \otimes \dots \otimes \mathbf{u}_N(:, r).$$

However, determining the rank of a tensor is an NP hard problem, so it is difficult to determine the value of R necessary to approximate the original tensor. The package computes this decomposition using the alternating least squares algorithm described by [Carroll and Chang \(1970\)](#) and [Harshman \(1970\)](#).

Higher-order singular value decomposition

The higher-order singular value decomposition (HOSVD) is a specific case of the Tucker decomposition with restrictions. The matrices $\mathbf{U}$ are unitary and the core tensor $\mathcal{B}$ has subtensors, defined as setting the n th index to a value α , that are orthogonal ([De Lathauwer et al., 2000](#)). Additionally, the subtensors have a descending order of Frobenius norms:

$$\|\mathcal{B}_{i_n=1}\| \geq \|\mathcal{B}_{i_n=2}\| \geq \dots \geq \|\mathcal{B}_{i_n=I_n}\| \geq 0.$$

These Frobenius norms are the n -mode singular values of $\mathcal{A}$, and the vector $U_i(n)$ is the i th n -mode singular vector.

Tensor-Train decomposition

The Tensor-Train decomposition factorizes an N -th order tensor $\mathcal{A}$ in a product of third-order tensors ([Oseledets, 2011](#)). These are denoted as $\mathbf{U}_1, \mathcal{U}_2, \dots, \mathbf{U}_N$ and are called TT-cores:

$$\mathcal{A}(i_1, \dots, i_N) = \mathbf{U}_1(i_1, :) \mathcal{U}_2(:, i_2, :) \mathcal{U}_3(:, i_3, :) \dots \mathbf{U}_N(:, i_N).$$

The package computes this decomposition using the TT-SVD algorithm. The user can provide the ranks directly or allow them to be selected using an approximation accuracy.

5 Model fitting and inference

One major functionality that `tensormodels` provides is MLE estimation. This is done via two different algorithms: for the tensor variate normal distribution, a flip-flop algorithm is used. For tensor variate skewed distributions, an EM algorithm is used.

Before discussing the tensor variate normal, we discuss the simpler matrix variate normal distribution. The matrix variate normal distribution can be considered a special case of the multivariate normal distribution, where we force the restriction that the covariance matrix is separable. That is, the covariance of the vectorized matrix draws can be described as the Kronecker product of two matrices.

Let $\mathbf{X} \sim N(\mathbf{M}, \mathbf{U}, \mathbf{V}) \in \mathbb{R}^{n \times p}$ and set $\mathbf{y} = \text{vec}(\mathbf{X})$. The matrix variate normal distribution imposes the separable covariance restriction

$$\Sigma = \mathbf{V} \otimes \mathbf{U},$$

where $\mathbf{U}$ is the row covariance matrix and $\mathbf{V}$ is the column covariance matrix. Therefore

$$\text{vec}(\mathbf{X}) \sim N_{np}(\text{vec}(\mathbf{M}), \mathbf{V} \otimes \mathbf{U}).$$

This is a nonlinear restriction on the natural parameter space of the full multivariate normal family. For the matrix variate normal, the covariance matrix is restricted to be defined

as the Kronecker product of two covariance matrices. This makes its parameter space a lower-dimensional curved subset of the full multivariate normal distribution (Roś et al., 2016).

By default, the initial values for the parameters are chosen in different ways. The location parameter $\mathcal{M}$ is started as the median of the draws. This is because the mean of the draws is dependent on the skew and parameters. Each covariance matrix is initialized by centering by the median, matricizing along the corresponding dimension, and adding up the scatter matrix across all of the draws. This total is normalized to have a trace equal to its dimension to ensure the scale is not disproportionate. Initial values for the parameters are set to be conservative and specific to the distribution. For example, for the skewed t , $\nu = 20$. Finally, the skew parameter $\mathcal{A}$ is computed as a function of the expected value of the draws, $\mathcal{M}$, and the relevant parameters for that distribution.

The individual covariance matrices in a Kronecker product are not defined uniquely. Multiplying one covariance matrix by a constant and another by the inverse of that constant produces the same full covariance matrix. The package addresses this by normalizing the trace of each covariance matrix except the final one to equal its corresponding mode dimension. This preserves the overall Kronecker covariance while giving the individual mode covariance estimates a consistent scale.

For the matrix variate normal model, differentiating the likelihood gives the updates

$$\hat{\mathbf{V}} = \frac{1}{Np} \sum_{i=1}^N \mathbf{x}_{ic}^T \hat{\mathbf{U}}^{-1} \mathbf{x}_{ic}$$

and

$$\hat{\mathbf{U}} = \frac{1}{Nn} \sum_{i=1}^N \mathbf{x}_{ic} \hat{\mathbf{V}}^{-1} \mathbf{x}_{ic}^T.$$

Each estimate depends on the other, which leads to the iterative flip-flop algorithm. If $N > \max(n, p)$, then the flip-flop algorithm has only one solution and the MLEs are unique under the conditions described by Srivastava et al. (2009).

Convergence checks are done by comparing the change in likelihood relative to the previous iteration. At the t -th iteration, the equation used is

$$A = \frac{\loglik_t - \loglik_{t-1}}{\loglik_{t-1}}$$

The algorithm ends when $A < \epsilon$, and by default $\epsilon = 1e - 6$.

Explain estimation, initialization, convergence checks, uncertainty quantification, and any constraints or identifiability conventions.

6 Diagnostic Tests

We assess tensor variate normality using the Mahalanobis distance. If $\mathcal{X}$ follows an order- O tensor variate normal distribution, then $\text{vec}(\mathcal{X})$ follows a multivariate normal distribution with mean $\text{vec}(\mathcal{M})$ and covariance matrix $\Sigma_O \otimes \cdots \otimes \Sigma_1$. This is why we can vectorize the tensor draws and compare the multivariate Mahalanobis distances to the tensor variate Mahalanobis distances. If we have properly reconstructed the structure using the MLEs $\hat{\mathcal{M}}$ and $\hat{\Sigma}_O, \dots, \hat{\Sigma}_1$, then the standardized draws should be from a standard normal.

If a tensor normal covariance structure is present, then the multivariate and tensor variate distances should be close to one another. Following the matrix variate normality paper, this can be visualized with a distance-distance plot: when the tensor normal structure is appropriate, the distances lie close to the reference line; when the Kronecker product structure is absent, the distances diverge.

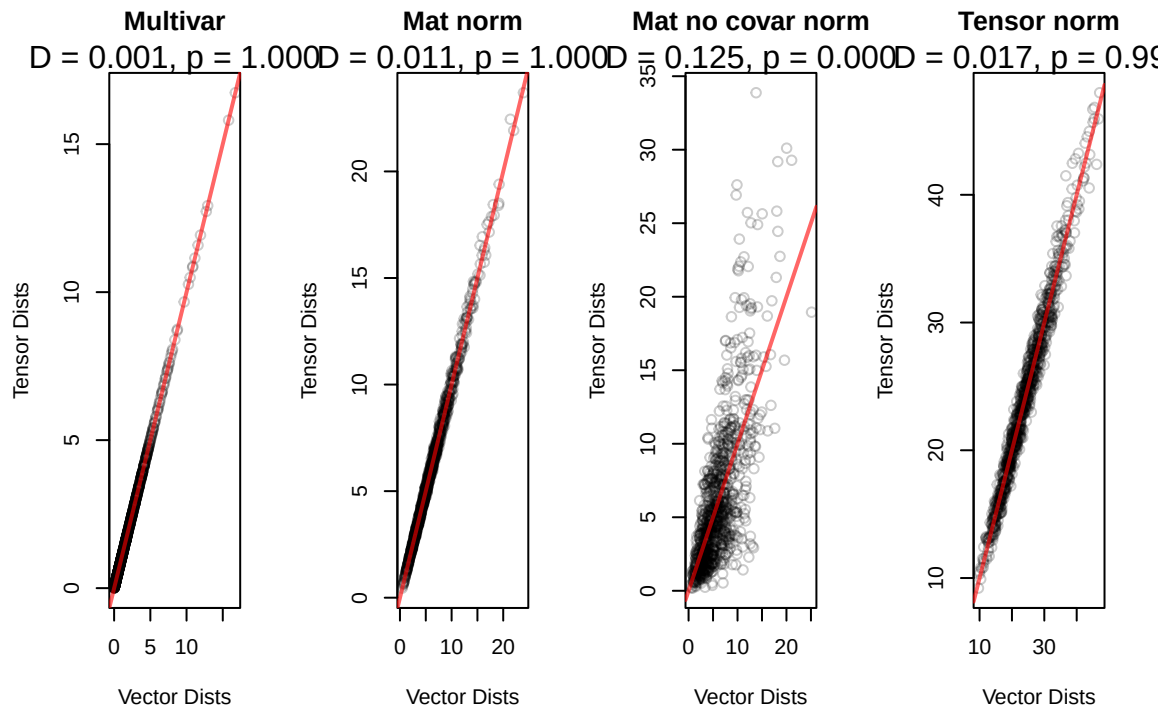


Figure 2: From left to right, the input data is a multivariate normal, matrix variate normal with a separable covariance structure, matrix variate with nonseparable covariance matrices, and tensor variate normal with a separable covariance structure. When there is a separable covariance structure, the tensor variate distances should be comparable to the vector distances, leading to a small D value and a large p -value.

The function `mahalanobis_dist()` computes both distances using fitted MLEs, and `mahalanobis_test()` performs a two-sample Kolmogorov-Smirnov test to compare their empirical distributions.

The figure below shows the Mahalanobis distance-distance plots along with the KS test statistic and p -value.

We are also interested in performing a likelihood ratio test to see if one of the covariance matrices might be the identity matrix. Thus, we run the algorithm and restrict these matrices.

For a restriction set $\mathcal{R} \subset \{1, \dots, D\}$, the restricted model fixes $\Sigma_r = I_{n_r}$ for every $r \in \mathcal{R}$, while all other covariance matrices are estimated. The likelihood ratio test compares

$$H_0 : \Sigma_r = I_{n_r} \forall r \in \mathcal{R} \quad \text{vs.} \quad H_A : \Sigma_r \neq I_{n_r} \text{ for at least one } r \in \mathcal{R}.$$

The test statistic is

$$2\{\ell(\hat{\theta}_{\text{full}}) - \ell(\hat{\theta}_{\mathcal{R}})\} \sim \chi^2_{\nu_{\mathcal{R}}}, \quad \nu_{\mathcal{R}} = \sum_{r \in \mathcal{R}} \left\{ \frac{n_r(n_r + 1)}{2} - 1 \right\}$$

If the p -value is large, then we fail to reject the null hypothesis and conclude the restricted model fits the data as well as the unrestricted model. If the p -value is small, we reject the null hypothesis, which states the unrestricted model has a significantly better fit than the restricted model.

As expected, all of the restrictions lead to a model that is significantly different from the full model. All the covariance matrices are not the diagonal to best model the draws.

```
res <- map_dfr(list(NULL, c(1), c(2), c(3), c(1, 2), c(1, 3), c(2, 3)),
  draws = tvn_draws, test_restrict)
```

```
knitr::kable(
```

Table 3: LRT for Full Tensor Variate

restrict	statistic	df	p-val
None	0.00	0	1
1	13257.02	2	0
2	61541.52	5	0
3	163633.38	9	0
1, 2	75108.59	7	0
1, 3	178430.32	11	0
2, 3	226430.11	14	0

Table 4: LRT for Second Sigma Identity

restrict	statistic	df	p-val
None	0.000	0	1.000
1	12103.247	2	0.000
2	9.138	5	0.104
3	47443.558	9	0.000
1, 2	12119.269	7	0.000
1, 3	59436.610	11	0.000
2, 3	47461.651	14	0.000

```
res,
digits = 3,
escape = FALSE,
booktabs = TRUE,
caption = "LRT for Full Tensor Variate")
```

Now, we generate data where the second covariance matrix is diagonal. The model that restricts the 2nd covariance to be diagonal has a $p - val > 0.05$, meaning it does not have a significantly smaller likelihood than the full unrestricted model.

7 Performance and validation

7.1 Parameter recovery

To validate the distributions, we can compare the MLEs to the true values used to generate draws.

```
mean((tvn_mles$mu - mu_true)^2)
#> [1] 0.04388826

for(i in 1:3) {
  true_scaled <- sigmas_true[[i]] / sum(diag(sigmas_true[[i]]))
  est_scaled <- tvn_mles$sigmas[[i]] / sum(diag(tvn_mles$sigmas[[i]]))

  print(mean((est_scaled - true_scaled)^2))
}
#> [1] 1.348412e-06
#> [1] 2.30351e-06
#> [1] 7.609639e-06
```

8 Examples

Now that we have set up our tensor distributions, we can use them on a real-world example. This data set is called “Condition monitoring of hydraulic systems” from the UC Irvine Machine Learning Repository. It has data from 6 sensors that measure pressure. Each pressure sensor is represented as a text file containing 2205 cycles and 6000 measurements, where the 6000 is 100 measurements per second taken over 60 seconds.

```
hydraulic_dir <- if (dir.exists("hydraulic_systems")) {  
  "hydraulic_systems"  
} else {  
  "paper/hydraulic_systems"  
}  
  
profile <- read.table(file.path(hydraulic_dir, "profile.txt"))  
  
names(profile) <- c("cooler",  
                   "valve",  
                   "pump_leakage",  
                   "accumulator",  
                   "stable")  
  
optimal_ids <- which( # find similar cycles  
  profile$cooler == 100 &  
  profile$valve >= 90 &  
  profile$pump_leakage == 0 &  
  profile$accumulator >= 115 &  
  profile$stable == 0)  
  
all_ps <- array(dim = c(6, 40, 60, 100))  
  
for(i in 1:6) { # read in data from 6 sensors  
  file_curr <- file.path(hydraulic_dir, paste0("PS", i, ".txt"))  
  ps_curr <- as.matrix(read.table(file_curr))  
  
  for (j in 1:40) {  
    curr_id <- optimal_ids[j]  
    all_ps[i, j, , ] <- matrix(ps_curr[curr_id, ],  
                              nrow = 60, ncol = 100,  
                              byrow = TRUE)  
  }  
}  
  
perm_ps <- aperm(all_ps, c(2, 1, 3, 4))  
  
# take 50 measurements per second instead of 100, too many data points  
perm_ps <- perm_ps[, , , seq(1, 100, by = 2)]  
  
pressure_list <- vector("list", dim(perm_ps)[1])  
  
for (k in 1:dim(perm_ps)[1]) { # store as list for tensor analysis  
  pressure_list[[k]] <- perm_ps[k, , ,]  
}  
  
perm_ps_mat <- array(perm_ps, dim = c(40, 6, 60 * 50))
```

```

press_mat <- vector("list", dim(perm_ps_mat)[1])

for (k in seq_along(press_mat)) { # store as list for matrix analysis
  press_mat[[k]] <- perm_ps_mat[k, , , drop = TRUE]
}

```

We are only interested in modeling when the system is working properly, so after filtering for that condition, we are left with 40 cycles. We assume that these 40 cycles are independent, identically distributed draws - that is, each of these cycles represents the same distribution of pressure values whenever the hydraulic test rig is working as intended. We also split the column containing 6000 measurements into a matrix of 60×50 , to represent the covariance structure of the repeated measurements taken within a second.

Now we can fit a tensor variate normal and a matrix variate normal to the data. The log likelihood for the tensor variate model is much higher than the matrix variate, and its BIC is also much lower as it estimates less parameters than the matrix variate model. This shows a situation where restructuring the data into a tensor can lead to much better model fit results.

```

# tensor and matrix models
res_norm_tensor <- tensor_mle(pressure_list, model = "normal", quiet = FALSE)
res_norm_matrix <- tensor_mle(press_mat, model = "normal", quiet = FALSE)

tibble(model = c("Tensor", "Matrix"),
       loglik = c(res_norm_tensor$loglik, res_norm_matrix$loglik),
       BIC = c(res_norm_tensor$BIC, res_norm_matrix$BIC),
       model_obj = list(res_norm_tensor, res_norm_matrix))

plot_matrix(cov2cor(res_norm_tensor$sigmas[[1]])) +
  labs(title = expression(R[1])) +
plot_matrix(cov2cor(res_norm_tensor$sigmas[[2]])) +
  labs(title = expression(R[2])) +
plot_matrix(cov2cor(res_norm_tensor$sigmas[[3]])) +
  labs(title = expression(R[3]))

model_compare <- function(draws) {
  res_normal <- tensor_mle(draws, model = "normal", quiet = FALSE)
  res_skewt <- tensor_mle(draws, model = "skewt", quiet = FALSE)
  res_vargam <- tensor_mle(draws, model = "vargamma", quiet = FALSE)
  res_invgauss <- tensor_mle(draws, model = "invgauss", quiet = FALSE)
  res_genhyper <- tensor_mle(draws, model = "genhyper", quiet = FALSE)

  all_mod <-
    tibble(model = c("Normal", "Skewt", "Vargamma", "Invgauss", "Genhyper"),
          loglik = c(res_normal$loglik, res_skewt$loglik,
                    res_vargam$loglik, res_invgauss$loglik, res_genhyper$loglik),
          k = c(res_normal$k, res_skewt$k,
                res_vargam$k, res_invgauss$k, res_genhyper$k),
          AIC = c(res_normal$AIC, res_skewt$AIC,
                  res_vargam$AIC, res_invgauss$AIC, res_genhyper$AIC),
          BIC = c(res_normal$BIC, res_skewt$BIC,
                  res_vargam$BIC, res_invgauss$BIC, res_genhyper$BIC),
          mod_obj = list(res_normal, res_skewt, res_vargam, res_invgauss, res_genhyper)
    )

  all_mod
}

```

```
res <- model_compare(pressure_list)

best_mod <- (res |> arrange(BIC) |> slice_head(n = 1))$mod_obj[[1]]
```

In this case, the best model was the variance gamma.

```
normal_mean <- res_norm_tensor$mu

vargam_mean <- best_mod$mu + best_mod$skew

plot_matrix(best_mod$mu[1, , ]) +
  labs(title = "VG fitted mean, sensor 1")

plot_matrix(normal_mean[1, , ]) +
  labs(title = "Normal fitted mean, sensor 1") +
plot_matrix(vargam_mean[1, , ]) +
  labs(title = "VG fitted mean, sensor 1")

plot_matrix(vargam_mean[1, , ] - normal_mean[1, , ]) +
  labs(title = "VG mean - normal mean, sensor 1")

plot_matrix(cov2cor(best_mod$sigmas[[1]])) +
  labs(title = expression(R[1])) +
plot_matrix(cov2cor(best_mod$sigmas[[2]])) +
  labs(title = expression(R[2])) +
plot_matrix(cov2cor(best_mod$sigmas[[3]])) +
  labs(title = expression(R[3]))
```

9 Discussion

Summarize the package's strengths, limitations, and intended scope. Mention important future work without turning this section into a feature wish list.

10 Reproducibility

List any external data and scripts required to reproduce the article. Place supporting code in `scripts/`, data in `data/`, and figure assets in `figures/`.

```
#> R version: 4.5.2
#> Platform: aarch64-apple-darwin20
#> rjtools version: 1.0.21
```

11 Acknowledgments

Add funding, contributor, and institutional acknowledgments here.

12 Appendix

12.1 Derivation of EM

The idea of EM is that, if we had the complete data, maximizing the complete likelihood would be simpler to account for the missing data. In this case, we simulate our tensor variate normal variance-mean mixtures as $\mathcal{X} = \mathcal{M} + W\mathcal{A} + \sqrt{W}\mathcal{V}$.

If we knew the latent draws W , then the distribution of $\mathcal{X}$ would be normal. More specifically $X|W \sim N(\mathcal{M} + w\mathcal{A}, w \otimes_{d=1}^D \Sigma_d)$. We are interested in maximizing the joint probability function $f(X, W) = f(X|W)f(W)$.

In the expectation step, we will estimate the w values that are most likely.

Earlier, we stated the probability distribution for a tensor variate normal. Now, we state the distribution for $f(\mathcal{X}|w)$.

$$f(\mathcal{X}|w) = (2\pi)^{-n^*/2} \left(\prod_{d=1}^D |\Sigma_d|^{-n^*/(2n_d)} \right) \exp \left\{ -\frac{1}{2} \text{vec}(\mathcal{X} - \mathcal{M} - w\mathcal{A})^T \left(\otimes_{d=1}^D \Sigma_d^{-1} \right) \text{vec}(\mathcal{X} - \mathcal{M} - w\mathcal{A}) \right\}.$$

And so,

$$\ell(\theta; \mathcal{X}, w) = \log f(\mathcal{X}|w) + \log f(w).$$

12.2 Proof of Flip-Flop for TVN

Let $\mathcal{X}_1, \dots, \mathcal{X}_K$ be IID from

$$TVN_{n_1, n_2, n_3}(\mathcal{M}, \mathbf{U}_1, \mathbf{U}_2, \mathbf{U}_3),$$

or equivalently,

$$\text{vec}(\mathcal{X}_k) \sim N(\text{vec}(\mathcal{M}), \mathbf{U}_3 \otimes \mathbf{U}_2 \otimes \mathbf{U}_1).$$

Let $\mathcal{Y}_k = \mathcal{X}_k - \mathcal{M}$, let $n_{-j} = \prod_{\ell \neq j} n_\ell$, and define

$$\mathbf{U}_{-1} = \mathbf{U}_3 \otimes \mathbf{U}_2, \quad \mathbf{U}_{-2} = \mathbf{U}_3 \otimes \mathbf{U}_1, \quad \mathbf{U}_{-3} = \mathbf{U}_2 \otimes \mathbf{U}_1.$$

Up to a constant that does not depend on the parameters, the log-likelihood can be written using the mode-1 unfolding as

$$\begin{aligned} \ell = & -\frac{K}{2} [n_2 n_3 \log |\mathbf{U}_1| + n_1 n_3 \log |\mathbf{U}_2| + n_1 n_2 \log |\mathbf{U}_3|] \\ & -\frac{1}{2} \sum_{k=1}^K \text{tr} \left[\mathbf{U}_1^{-1} \mathcal{Y}_{k(1)} (\mathbf{U}_3 \otimes \mathbf{U}_2)^{-1} \mathcal{Y}_{k(1)}^T \right]. \end{aligned}$$

More generally, using mode- j matricization, the quadratic term is

$$\text{tr} \left[\mathbf{U}_j^{-1} \mathcal{Y}_{k(j)} \mathbf{U}_{-j}^{-1} \mathcal{Y}_{k(j)}^T \right],$$

which is equivalent to the tensor inner-product notation

$$\left\langle \mathcal{Y}_k, \mathcal{Y}_k \times_1 \mathbf{U}_1^{-1} \times_2 \mathbf{U}_2^{-1} \times_3 \mathbf{U}_3^{-1} \right\rangle.$$

In vectorized form, let

$$\mathbf{\Omega} = \mathbf{U}_3 \otimes \mathbf{U}_2 \otimes \mathbf{U}_1.$$

The part of the log-likelihood involving $\mathcal{M}$ is

$$-\frac{1}{2} \sum_{k=1}^K [\text{vec}(\mathcal{X}_k) - \text{vec}(\mathcal{M})]^T \mathbf{\Omega}^{-1} [\text{vec}(\mathcal{X}_k) - \text{vec}(\mathcal{M})].$$

Differentiating with respect to $\text{vec}(\mathcal{M})$ gives

$$\frac{\partial \ell}{\partial \text{vec}(\mathcal{M})} = \mathbf{\Omega}^{-1} \sum_{k=1}^K [\text{vec}(\mathcal{X}_k) - \text{vec}(\mathcal{M})].$$

Setting this equal to zero gives

$$\hat{\mathcal{M}} = \bar{\mathcal{X}} = \frac{1}{K} \sum_{k=1}^K \mathcal{X}_k.$$

Now hold the other covariance matrices fixed and estimate $\mathbf{U}_j$. Define

$$\mathbf{S}_j = \sum_{k=1}^K \mathcal{Y}_{k(j)} \mathbf{U}_{-j}^{-1} \mathcal{Y}_{k(j)}^T.$$

The part of the log-likelihood involving $\mathbf{U}_j$ is

$$\ell_j = -\frac{Kn-j}{2} \log |\mathbf{U}_j| - \frac{1}{2} \text{tr} \left(\mathbf{U}_j^{-1} \mathbf{S}_j \right).$$

Using

$$d \log |\mathbf{U}_j| = \text{tr}(\mathbf{U}_j^{-1} d\mathbf{U}_j)$$

and

$$d \text{tr}(\mathbf{U}_j^{-1} \mathbf{S}_j) = -\text{tr} \left(\mathbf{U}_j^{-1} d\mathbf{U}_j \mathbf{U}_j^{-1} \mathbf{S}_j \right),$$

the differential is

$$\begin{aligned} d\ell_j &= -\frac{Kn-j}{2} \text{tr}(\mathbf{U}_j^{-1} d\mathbf{U}_j) \\ &\quad + \frac{1}{2} \text{tr} \left(\mathbf{U}_j^{-1} d\mathbf{U}_j \mathbf{U}_j^{-1} \mathbf{S}_j \right). \end{aligned}$$

Moving $d\mathbf{U}_j$ to the end of each trace gives

$$d\ell_j = \frac{1}{2} \text{tr} \left[\left(-Kn-j \mathbf{U}_j^{-1} + \mathbf{U}_j^{-1} \mathbf{S}_j \mathbf{U}_j^{-1} \right) d\mathbf{U}_j \right].$$

Since this must be zero for all $d\mathbf{U}_j$,

$$-Kn-j \mathbf{U}_j^{-1} + \mathbf{U}_j^{-1} \mathbf{S}_j \mathbf{U}_j^{-1} = \mathbf{0}.$$

Multiplying on the left and right by $\mathbf{U}_j$ gives

$$\hat{\mathbf{U}}_j = \frac{1}{Kn-j} \mathbf{S}_j.$$

Substituting back the definition of $\mathbf{S}_j$ gives the three TVN updates:

$$\hat{\mathbf{U}}_1 = \frac{1}{Kn_2 n_3} \sum_{k=1}^K (\mathcal{X}_k - \bar{\mathcal{X}})_{(1)} (\hat{\mathbf{U}}_3 \otimes \hat{\mathbf{U}}_2)^{-1} (\mathcal{X}_k - \bar{\mathcal{X}})_{(1)}^T,$$

$$\hat{\mathbf{U}}_2 = \frac{1}{Kn_1 n_3} \sum_{k=1}^K (\mathcal{X}_k - \bar{\mathcal{X}})_{(2)} (\hat{\mathbf{U}}_3 \otimes \hat{\mathbf{U}}_1)^{-1} (\mathcal{X}_k - \bar{\mathcal{X}})_{(2)}^T,$$

and

$$\hat{\mathbf{U}}_3 = \frac{1}{Kn_1 n_2} \sum_{k=1}^K (\mathcal{X}_k - \bar{\mathcal{X}})_{(3)} (\hat{\mathbf{U}}_2 \otimes \hat{\mathbf{U}}_1)^{-1} (\mathcal{X}_k - \bar{\mathcal{X}})_{(3)}^T.$$

Because each covariance update depends on the other two covariance estimates, the MLEs are found by iterating these equations. The individual U_j are determined only up to scale constraints; the Kronecker product is the identifiable covariance structure.

References

- B. W. Bader and T. G. Kolda. Algorithm 862: MATLAB tensor classes for fast algorithm prototyping. *ACM Transactions on Mathematical Software*, 32(4):635–653, 2006. doi: 10.1145/1186785.1186794. [p6]
- B. W. Bader and T. G. Kolda. Efficient MATLAB computations with sparse and factored tensors. *SIAM Journal on Scientific Computing*, 30(1):205–231, 2007. doi: 10.1137/060676489. [p6]
- B. W. Bader, T. G. Kolda, et al. *Tensor Toolbox for MATLAB*, 2025. URL <https://www.tensortoolbox.org/>. Version 3.8. [p6]
- O. Barndorff-Nielsen, J. Kent, and M. Sørensen. Normal variance-mean mixtures and z distributions. *International Statistical Review*, 50(2):145–159, 1982. [p5]
- J. D. Carroll and J.-J. Chang. Analysis of individual differences in multidimensional scaling via an n-way generalization of eckart-young decomposition. *Psychometrika*, 35(3):283–319, 1970. doi: 10.1007/BF02310791. [p10]
- L. De Lathauwer, B. De Moor, and J. Vandewalle. A multilinear singular value decomposition. *SIAM Journal on Matrix Analysis and Applications*, 21:1253–1278, 2000. doi: 10.1137/S0895479896305696. [p10]
- R. A. Harshman. Foundations of the parafac procedure: Models and conditions for an explanatory multimodal factor analysis. 1970. [p10]
- F. L. Hitchcock. The expression of a tensor or a polyadic as a sum of products. *Journal of Mathematics and Physics*, 6(1-4):164–189, 1927. doi: 10.1002/sapm192761164. [p9]
- J. Kossaifi, Y. Panagakis, A. Anandkumar, and M. Pantic. Tensorly: Tensor learning in Python. *Journal of Machine Learning Research*, 20(26):1–6, 2019. URL <https://jmlr.org/papers/v20/18-277.html>. [p6]
- J. Li, J. Bien, and M. T. Wells. *rTensor: Tools for Tensor Analysis and Decomposition*, 2024a. URL <https://cran.r-project.org/package=rTensor>. R package version 1.4.9. [p6]
- J. Li, J. Bien, M. T. Wells, H. Xiao, and Y. Chen. *ttTensor: Tensor Train Decomposition*, 2024b. URL <https://cran.r-project.org/package=ttTensor>. R package version 1.0.1. [p6]
- I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5): 2295–2317, 2011. doi: 10.1137/090752286. [p10]
- L. Periša. *TensorToolbox.jl: Tensor Decompositions in Julia*, 2023. URL <https://github.com/lanaperisa/TensorToolbox.jl>. GitHub repository. [p6]
- pyttb developers. *pyttb: Python Tensor Toolbox*, 2026. URL <https://pyttb.readthedocs.io/>. Documentation accessed 2026-06-30. [p6]
- B. Roś, F. Bijma, J. C. de Munck, and M. C. M. de Gunst. Existence and uniqueness of the maximum likelihood estimator for models with a kronecker product covariance structure. *Journal of Multivariate Analysis*, 143:345–361, 2016. doi: 10.1016/j.jmva.2015.05.019. [p11]
- M. S. Srivastava, T. von Rosen, and D. von Rosen. Estimation and testing in general multivariate linear models with kronecker product covariance structure. *Sankhyā: The Indian Journal of Statistics, Series A*, 71(2):137–163, 2009. [p11]

TensorOperations.jl developers. *TensorOperations.jl*, 2026. URL <https://quantumkithub.github.io/TensorOperations.jl/stable/>. Documentation accessed 2026-06-30. [p6]

Anupama Kannan
Baylor University
Department of Statistics
Waco, Texas
ORCID: 0000-0003-2789-6997
Anupama_Kannan1@baylor.edu

David Kahle
Baylor University
Department of Statistics
Waco, Texas
ORCID: 0000-0002-9999-1558
David_Kahle@baylor.edu

Michael Gallagher
Baylor University
Department of Statistics
Waco, Texas
ORCID: 0000-0002-5487-8965
Michael_Gallagher@baylor.edu